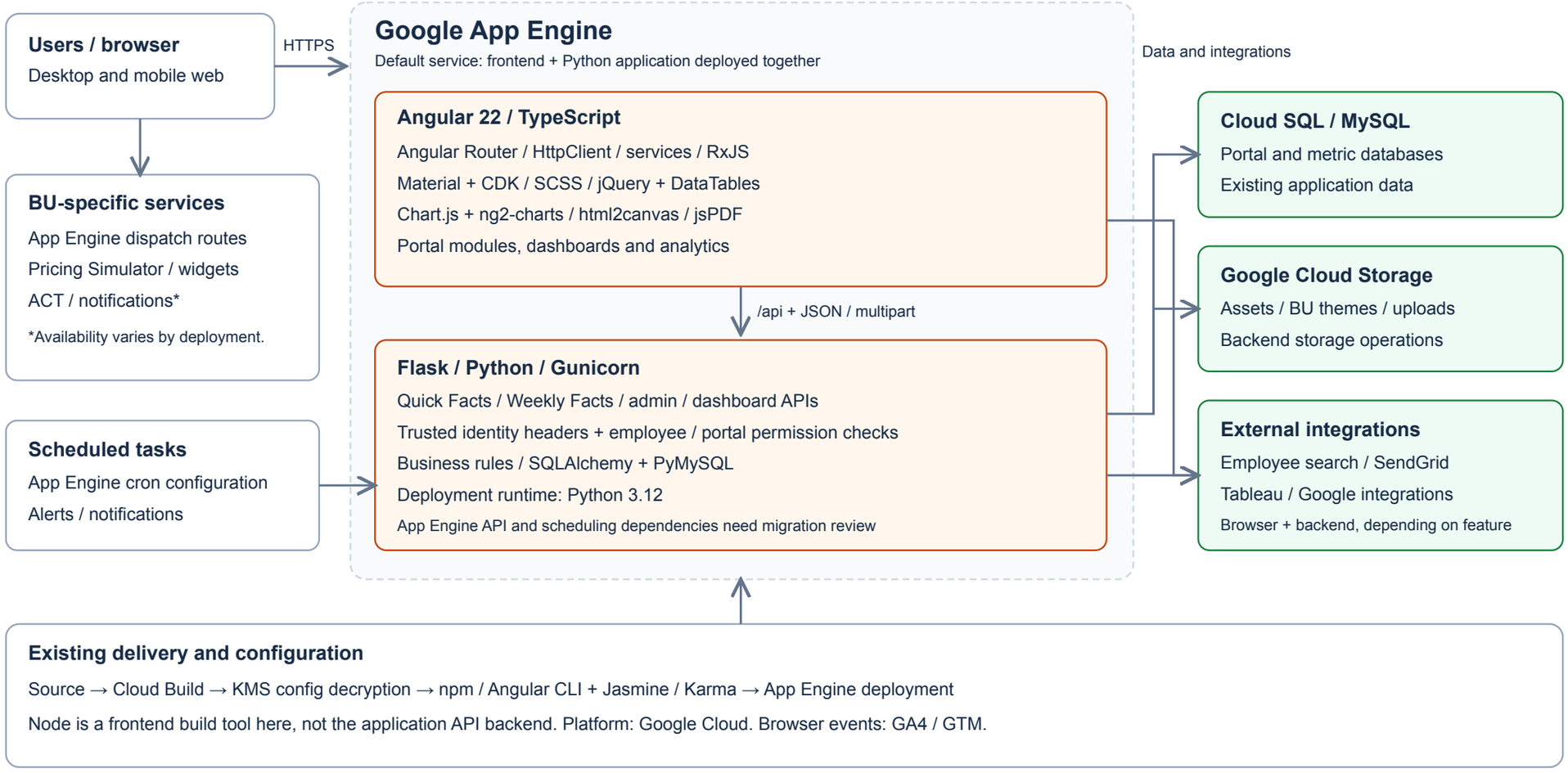
OLD Angular + Flask on App Engine

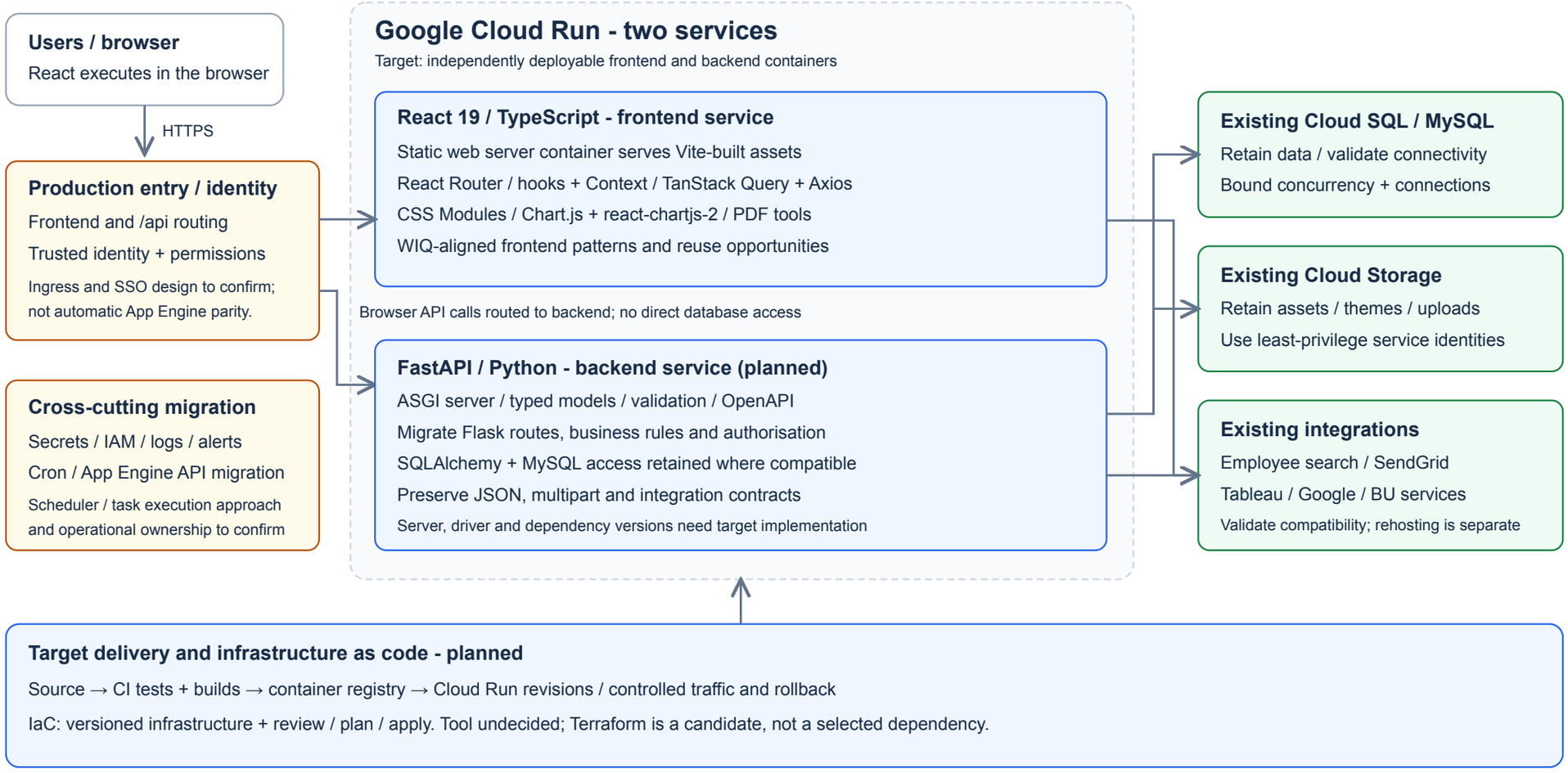
Existing application: browser frontend and Python APIs share the default App Engine deployment.



Established business rules, data and integrations remain valuable. "OLD" identifies the existing implementation, not obsolete technology.

NEW TARGET React + FastAPI on Cloud Run

Separate frontend and backend services; retain existing data/integration investments and introduce infrastructure as code.



Technology change Retain and validate Decision or readiness work

Target, not current deployment. IaC tool undecided; Terraform is a candidate. Identity, ingress and operational migration need design.

What changes, and what stays

Modernise the frontend, API framework and compute layer without adding a database rewrite.

Layer	OLD	NEW TARGET	Why it matters
Frontend	Angular / TypeScript	React / TypeScript	Align with WIQ's predominant frontend framework.
UI / data state	Material/CDK, services, RxJS, jQuery tables	React components, CSS Modules, Query + Axios	Reusable patterns and explicit API-state handling.
Python APIs	Flask	FastAPI / typed models / OpenAPI	Clearer contracts and validation; migrate business/access rules.
Hosting	App Engine app + dispatch	Cloud Run frontend + backend containers	Independent releases and container-based operations.
Infrastructure	App/dispatch/cron YAML + deploy scripts	Infrastructure as code, tool undecided	Reviewable resource changes and reproducible environments.
Build / tests	Angular CLI / Jasmine + Karma	Vite / Vitest + Testing Library / API tests	React-aligned tooling and planned container quality gates.
Retained	Python, MySQL, Cloud Storage, charts, integrations	Retain where compatible; reconnect and verify	Preserve data and business investment; reduce migration scope.

This is **not** a Node API rewrite. Node builds the frontend; Python remains the backend language. Cloud Build is retained with a planned container release pipeline.

WHY THIS MATTERS FOR WIQ

One frontend ecosystem. Broader team ownership.

Most WIQ projects already use React.

Aligning Store Insights creates opportunities to reuse team knowledge, components and delivery practices instead of maintaining a separate frontend skill silo.

Developer mobility

React-experienced teammates can move between projects with less framework-specific onboarding.

Shared UI investment

Reuse compatible WIQ layouts, filters, accessible components and test patterns once standards are agreed.

Cross-project support

A larger familiar contributor pool can spread review, maintenance and support responsibilities.

Clear API contracts

FastAPI models and OpenAPI support consistent integration, validation and contract testing.

Independent releases

Separate containers let frontend and backend evolve and scale independently, with versioned rollback.

Repeatable environments

Reviewed IaC definitions make infrastructure changes visible and reduce manual environment drift.

Measure, do not assume: onboarding time, cross-project contributions, component reuse, delivery lead time, deployment failures/recovery, p95 latency and cloud spend.

A target architecture, not a completed migration

Today: React still uses Flask/App Engine. Frontend identity is a demo stub; several portal routes remain placeholders. FastAPI, Cloud Run and IaC are planned.

1. Identity + ingress

Choose trusted SSO and frontend/API routing. Preserve portal/role access; review cookies, CORS and CSRF. App Engine header assumptions need explicit replacement.

2. API + feature parity

Migrate Flask handlers and framework-specific code. Verify JSON, errors, uploads, PDFs and all portal integrations, not only Quick Facts.

3. Data + runtime

Prove SQL/Storage access, credentials and outbound integrations. Match Cloud Run scaling to database connection limits; review blocking Python I/O.

4. IaC + platform ownership

Select the tool, state/locking, environments, IAM and secret delivery. Inventory App Engine cron/APIs and decide scheduler/task replacements.

5. CI/CD + observability

Add frontend/API quality gates, container builds and an approved registry. Define logging, alerts, revision rollout and rollback; align with WIQ standards.

6. Controlled cutover

Demonstrate functional, visual and accessibility parity. Benchmark latency/cost, stage traffic and retain a tested fallback until acceptance.

Decision summary: React + FastAPI + separate Cloud Run services approved as the direction. IaC tooling, identity/ingress and operational detail remain open. No guaranteed cost reduction or automatic performance gain.